

تقرير المشروع الرسمي

محاكي جدولة وحدة المعالجة المركزية باستخدام هياكل البيانات المناسبة

CPU Scheduling Simulator Using Appropriate Data Structures

إعداد	مؤمن عبدالله الشامي
إشراف	د. بلال مرشد
لغة التنفيذ	C++17
العام الجامعي	2026 م
رابط المشروع	https://github.com/alimohammedabotaleb-hub/cpu-scheduling-simulation-ds

تنفيذ الخوارزميات باستخدام Array و Queue و Priority Queue و Linked List و Stack

الملخص

يعرض هذا المشروع محاكاة برمجية متكاملة لأربع من أشهر خوارزميات جدولة وحدة المعالجة المركزية، وهي: خوارزمية «القادِم أولاً يُخدم أولاً» (FCFS)، وخوارزمية «المهمة الأقصر أولاً» (SJF)، وجدولة الأولوية (Priority)، وخوارزمية «التناوب الدائري» (Round Robin). بُني المحاكى بلغة ++C، واعتمد بصورة صريحة على هياكل بيانات مناسبة تشمل المصفوفات، والطابور، وطابور الأولوية، والقائمة المرتبطة، والمكدس.

تُحفظ بيانات العمليات وعينة الاختبار داخل البرنامج، ولذلك يبدأ التنفيذ مباشرة من دون الحاجة إلى إدخال يدوي. ينفذ المحاكى الخوارزميات الأربع على مجموعة العمليات نفسها، ثم يعرض ترتيب التنفيذ، ومخطط جانِب، وزمن الإكمال، وزمن الانتظار، وزمن الدوران لكل عملية، إضافة إلى متوسط زمن الانتظار، ومتوسط زمن الدوران، ونسبة استخدام وحدة المعالجة المركزية. ويختتم التنفيذ بجدول مقارنة موحد يوضح الفروق بين نتائج الخوارزميات.

يهدف المشروع إلى الربط بين المفاهيم النظرية لجدولة المعالج وبين التطبيق العملي لهياكل البيانات، مع توضيح أثر سياسة اختيار العملية التالية في مؤشرات الأداء. كما يتضمن المشروع تنظيمًا مناسبًا لملفات المصدر والتوثيق تمهيدًا لحفظه ونشره في مستودع GitHub.

الكلمات المفتاحية، CPU Scheduling ، FCFS ، SJF ، Priority Scheduling ، Round Robin ، Queue ، Priority Queue ، Array ، Linked List ، Stack ، Waiting Time ، Turnaround Time ، CPU Utilization.

1. مقدمة

تعمل أنظمة التشغيل على إدارة موارد الحاسوب وتوزيعها بين البرامج والعمليات الجارية. وتُعد وحدة المعالجة المركزية أحد أهم هذه الموارد؛ إذ قد تكون عدة عمليات جاهزة للتنفيذ في الوقت نفسه، في حين لا تستطيع الوحدة أحادية المعالج تنفيذ أكثر من عملية واحدة في اللحظة نفسها. لذلك يعتمد نظام التشغيل على جدول المعالج لاختيار العملية التي تحصل على المعالج وفق سياسة محددة.

تؤثر سياسة الجدولة في سرعة استجابة النظام، وفي مدة انتظار العمليات، وفي الزمن الكلي اللازم لإكمالها، وفي مقدار الاستفادة من وحدة المعالجة. ولا توجد خوارزمية واحدة تحقق أفضل نتيجة في جميع الحالات؛ لأن جودة الخوارزمية ترتبط بطبيعة العمليات، وأوقات وصولها، ومدد تنفيذها، وأولوياتها، والهدف التشغيلي للنظام.

يقدم المشروع نموذجًا تطبيقيًا موحدًا يسمح بتشغيل الخوارزميات الأربع على بيانات متطابقة، ثم مقارنة نتائجها بالاعتماد على مؤشرات كمية واضحة. ويزر التصميم كذلك العلاقة بين كل خوارزمية وهيكل البيانات الأنسب لتنفيذها.

2. مشكلة المشروع

تتمثل المشكلة في وجود مجموعة عمليات تتنافس على وحدة معالجة مركزية واحدة. تمتلك كل عملية رقمًا تعريفياً، ووقت وصول، وزمن تنفيذ، وقيمة أولوية. ويلزم تحديد ترتيب تنفيذ هذه العمليات وفق عدة سياسات جدولة، ثم حساب نتائج كل سياسة ومقارنتها بصورة عادلة.

تعالج المحاكاة الأسئلة الآتية:

1. ما ترتيب تنفيذ العمليات في كل خوارزمية؟
2. كم تنتظر كل عملية قبل إكمال تنفيذها؟
3. ما الزمن الكلي الذي تقضيه كل عملية منذ وصولها حتى إكمالها؟
4. ما متوسط زمن الانتظار ومتوسط زمن الدوران لكل خوارزمية؟
5. ما نسبة الزمن الذي تكون فيه وحدة المعالجة مشغولة؟
6. ما أثر هيكل البيانات المستخدم في طريقة اختيار العملية التالية؟

3. أهداف المشروع

يهدف المشروع إلى تحقيق ما يأتي:

1. محاكاة آلية عمل خوارزميات جدولة وحدة المعالجة المركزية.
2. تنفيذ FCFS و SJF و Priority Scheduling و Round Robin ضمن برنامج موحد.
3. توظيف المصفوفة والطاوير وطابور الأولوية والقائمة المرتبطة والمكدس توظيفًا واضحًا.
4. حساب زمن الانتظار وزمن الدوران ونسبة استخدام وحدة المعالجة المركزية.
5. عرض مخطط زمني يوضح تسلسل تنفيذ العمليات.
6. مقارنة أداء الخوارزميات على مجموعة بيانات واحدة.
7. توثيق افتراضات المحاكاة وقواعد كسر التعادل.
8. تنظيم ملفات المشروع ونشرها في مستودع GitHub لدعم إدارة الإصدارات وسهولة المراجعة.

4. نطاق المشروع

يشمل المشروع ما يأتي:

- محاكاة وحدة معالجة مركزية واحدة.
 - استخدام مجموعة عمليات محددة مسبقًا داخل الكود.
 - دعم أوقات وصول مختلفة وأزمنة تنفيذ وأولويات متعددة.
 - تنفيذ FCFS بصورة غير مستبقة.
 - تنفيذ SJF بصورة غير مستبقة.
 - تنفيذ Priority Scheduling بصورة غير مستبقة.
 - تنفيذ Round Robin بصورة مستبقة وفق شريحة زمنية ثابتة.
 - حساب CT و WT و TAT لكل عملية.
 - حساب متوسط WT ومتوسط TAT ونسبة CPU Utilization.
 - تمثيل فترات خمول المعالج عند عدم وجود عملية جاهزة.
 - عرض النتائج لكل خوارزمية، ثم عرض مقارنة ختامية.
- لا يحاكي المشروع تعدد الأنوية، أو المقاطعات الحقيقية لنظام التشغيل، أو كلفة التبديل السياقي، أو عمليات الإدخال والإخراج المتقطعة داخل العملية. وقد استُبعدت هذه العناصر للمحافظة على وضوح النموذج وتركيزه على مبادئ الجدولة وهياكل البيانات.

5. الأساس النظري

5.1 العملية Process

العملية هي برنامج قيد التنفيذ. وتُمثل كل عملية في المحاكى بالخصائص الآتية:

الرمز	الاسم	المعنى
PID	Process ID	المعرّف الفريد للعملية
AT	Arrival Time	الزمن الذي تصل عنده العملية إلى قائمة الجاهزين
BT	Burst Time	الزمن المطلوب لتنفيذ العملية على المعالج
PR	Priority	قيمة أولوية العملية
CT	Completion Time	الزمن الذي يكتمل عنده تنفيذ العملية
WT	Waiting Time	مجموع الزمن الذي انتظرته العملية في قائمة الجاهزين

الرمز	الاسم	المعنى
TAT	Turnaround Time	الزمن من وصول العملية حتى اكتمالها

5.2 قائمة الجاهزين Ready Queue

قائمة الجاهزين هي مجموعة العمليات التي وصلت وأصبحت مؤهلة للتنفيذ، لكنها تنتظر الحصول على وحدة المعالجة. تختلف طريقة تنظيم هذه القائمة واختيار عنصرها التالي باختلاف خوارزمية الجدولة.

5.3 الجدولة غير المستبقة والجدولة المستبقة

في الجدولة غير المستبقة تستمر العملية المختارة في استخدام المعالج حتى تنتهي؛ ومن أمثلتها النسخ المنفذة في المشروع من FCFS و SJF و Priority. أما الجدولة المستبقة فتسمح بإيقاف العملية قبل اكتمالها وإعادة إدخالها إلى قائمة الجاهزين، كما يحدث في Round Robin عند انتهاء الشريحة الزمنية.

5.4 معايير التقييم

تعتمد المقارنة في المشروع على ثلاثة مؤشرات رئيسية:

- متوسط زمن الانتظار: انخفاضه يعني أن العمليات تقضي وقتًا أقل في قائمة الجاهزين.
- متوسط زمن الدوران: انخفاضه يعني أن العمليات تكتمل في مدة أقصر منذ لحظة وصولها.
- استخدام وحدة المعالجة المركزية: ارتفاعه يعني تقليل فترات خمول المعالج خلال زمن المحاكاة.

6. تصميم النظام

6.1 مدخلات المحاكاة المدمجة

لا يعتمد البرنامج على إدخال يدوي أثناء التشغيل. تُعرّف مجموعة الاختبار والشريحة الزمنية داخل الكود، ثم تُحمّل آليًا عند بدء البرنامج.

العملية	وقت الوصول AT	زمن التنفيذ BT	الأولوية PR
P1	2	5	3
P2	3	2	1
P3	4	8	4
P4	5	3	2
P5	7	4	1

الشريحة الزمنية لخوارزمية Round Robin: 2 وحدة زمنية.

6.2 دورة عمل البرنامج

يمر التنفيذ بالمراحل الآتية:

- إنشاء العمليات من المصفوفة المدمجة داخل الكود.
- نقل بيانات العمليات إلى مخزن العمليات المرتبط.
- تمرير بيانات العمليات الأصلية بصفة ثابتة، وإنشاء نتيجة مستقلة لكل خوارزمية.
- تنفيذ FCFS وحفظ النتيجة.
- تنفيذ SJF وحفظ النتيجة.
- تنفيذ Priority Scheduling وحفظ النتيجة.

7. تنفيذ Round Robin وحفظ النتيجة.
8. حساب CT و WT و TAT والمتوسطات واستخدام المعالج.
9. عرض مخطط جانت وسجل التنفيذ لكل خوارزمية.
10. عرض جدول المقارنة النهائي.

6.3 وحدات النظام

الوحدة	المسؤولية
نموذج العملية	حفظ PID و AT و BT و PR ، مع استخدام موضعها الأصلي لكسر التعادل
ProcessLinkedList	حفظ جميع العمليات باستخدام قائمة مرتبطة
CircularQueue	إدارة FCFS و Round Robin وفق FIFO
MinHeapPriorityQueue	اختيار أقل BT في SJF أو أعلى أولوية في Priority
وحدة المصفوفات	حفظ البيانات الثابتة والنتائج والأزمنة المتبقية
SegmentStack	حفظ مقاطع التنفيذ وإظهارها بترتيب عكسي
وحدة الحساب	اشتقاق CT و WT و TAT والمتوسطات والاستخدام
وحدة العرض	طباعة الجداول ومخطط جانت والمقارنة

7. هياكل البيانات المستخدمة

7.1 المصفوفة Array

تستخدم المصفوفات الخام ثابتة الحجم لتعريف بيانات العينة، وحفظ مؤشرات العمليات، والأزمنة المتبقية، وأزمنة الإكمال، والنتائج. تتيح المصفوفة وصولاً مباشرًا إلى العنصر باستخدام الفهرس بزمان تقريبي ثابت $O(1)$ ، ويحدد الحد الأعلى للعناصر مسبقًا داخل البرنامج.

7.2 الطابور Queue

يعمل الطابور وفق قاعدة «الداخل أولاً يخرج أولاً». FIFO. ولهذا يناسب FCFS ، كما يمثل قائمة الجاهزين في Round Robin ؛ إذ تدخل العملية الجديدة في نهاية الطابور، وتخرج العملية التالية من مقدمته، وتُعاد العملية غير المكتملة إلى النهاية بعد انتهاء شريحتها الزمنية.

7.3 طابور الأولوية Priority Queue

يُنفذ طابور الأولوية باستخدام كومة ثنائية صغرى Binary Min-Heap في SJF تكون قيمة المفاضلة الأساسية هي زمن التنفيذ الأقل. وفي Priority Scheduling تكون قيمة المفاضلة الأساسية هي رقم الأولوية الأقل، حيث يعتمد المشروع القاعدة: الرقم الأصغر يعني أولوية أعلى. وعند التعادل يُستخدم وقت الوصول، ثم ترتيب تعريف العملية.

7.4 القائمة المرتبطة Linked List

تُستخدم الفئة `ProcessLinkedList` بوصفها المخزن الرئيس لبيانات العمليات. تتكون من عقد، وتحمل كل عقدة بيانات عملية ومؤشرًا إلى العقدة التالية. تسمح هذه البنية بإضافة العمليات من دون الحاجة إلى إزاحة عناصر أخرى، ثم تنسخ الدالة `copyToArray` محتوى العقد إلى مصفوفة خام ثابتة تستخدمها خوارزميات الجدولة.

7.5 المكدس Stack

تطبق الفئة SegmentStack قاعدة «الداخل أخيرًا يخرج أولًا» LIFO. تُدفع مقاطع التنفيذ إلى المكدس أثناء المحاكاة، ثم تُسترجع بالعكس لإظهار سجل التنفيذ العكسي. وجود المكدس لا يغير قرار الجدولة، لكنه يوثق استخدام هذه البنية ويقدم طريقة إضافية لاستعراض مسار التنفيذ.

7.6 ملائمة الهياكل للخوارزميات

الخوارزمية/المهمة	الهيكل الرئيس	سبب الاختيار
تخزين العينة والنتائج	Array	وصول مباشر وتنظيم متتابع للبيانات
تخزين العمليات	Linked List	إضافة عقد وحفظ مرّن للعمليات
FCFS	Queue	المحافظة على ترتيب الوصول
SJF	Priority Queue	استخراج أقصر عملية جاهزة بكفاءة
Priority Scheduling	Priority Queue	استخراج العملية الأعلى أولوية بكفاءة
Round Robin	Queue	تدوير العمليات وإعادة غير المكتمل إلى النهاية
سجل التنفيذ العكسي	Stack	استرجاع المقاطع بعكس ترتيب إدراجها

8. الخوارزميات المنفذة

8.1 خوارزمية FCFS

تنفذ FCFS العمليات بحسب ترتيب وصولها. تدخل العمليات الجاهزة إلى الطابور، وتخرج من مقدمته للتنفيذ. إذا كان الطابور فارغًا ولم تصل العملية التالية بعد، يسجل البرنامج فترة خمول حتى وقت وصولها.

قاعدة الاختيار: أقدم عملية وصلت أولًا. نوع الجدولة: غير مستبقة. هيكل البيانات الرئيس: Queue. ميزة أساسية: بساطة التنفيذ وعدالة ترتيب الوصول. قيد أساسي: قد يؤدي وصول عملية طويلة أولًا إلى زيادة انتظار العمليات القصيرة، ويُعرف ذلك بتأثير القافلة. Convoy Effect.

8.2 خوارزمية SJF

تختار SJF العملية ذات زمن التنفيذ الأقصر من بين العمليات التي وصلت وأصبحت جاهزة. لا يجوز اختيار عملية لم تصل بعد، حتى إن كان زمن تنفيذها أقصر. بعد بدء العملية المختارة تستمر حتى الإكمال لأن النسخة المعتمدة غير مستبقة.

قاعدة الاختيار: أقل Burst Time بين العمليات الجاهزة. نوع الجدولة: غير مستبقة. هيكل البيانات الرئيس: Priority Queue. ميزة أساسية: غالبًا ما تحقق متوسط انتظار منخفضًا عند معرفة أزمّة التنفيذ. قيد أساسي: قد تنتظر العمليات الطويلة مدة كبيرة إذا استمر وصول عمليات قصيرة.

8.3 خوارزمية Priority Scheduling

تختار هذه الخوارزمية العملية الأعلى أولوية من العمليات الجاهزة. اعتمد المشروع رقمًا أصغر للدلالة على أولوية أعلى؛ فالأولوية 1 أعلى من الأولوية 2. تُحسم حالات التعادل بوقت الوصول، ثم ترتيب تعريف العملية.

قاعدة الاختيار: أصغر رقم أولوية بين العمليات الجاهزة. نوع الجدولة: غير مستبقة. هيكل البيانات الرئيس: Priority Queue. ميزة أساسية: تمكين العمليات المهمة من التقدم على العمليات الأقل أهمية. قيد أساسي: احتمال تأخر العمليات منخفضة الأولوية، ويمكن الحد من ذلك في الأنظمة الواقعية باستخدام Aging.

8.4 خوارزمية Round Robin

تمنح Round Robin كل عملية جاهزة شريحة زمنية ثابتة تسمى Time Quantum. إذا انتهت العملية قبل نهاية الشريحة تُسجل مكتملة، وإلا تتوقف بعد استهلاك الشريحة وتُعاد إلى نهاية الطابور. تصاف العمليات التي وصلت أثناء الشريحة إلى الطابور قبل إعادة العملية الحالية غير المكتملة، بما يحافظ على ترتيب الوصول العادل.

قاعدة الاختيار: الدور التالي في الطابور الدائري. نوع الجدولة: مستبقة. هيكل البيانات الرئيس: Queue. ميزة أساسية: عدالة توزيع وقت المعالج وملاءمة الأنظمة التفاعلية. قيد أساسي: تؤثر قيمة Time Quantum في الأداء؛ فالقيمة الكبيرة تقربها من FCFS، والقيمة الصغيرة جدًا ترفع عدد التبديلات السياقية في النظام الحقيقي.

9. المعادلات المستخدمة

9.1 زمن الدوران

زمن الدوران هو الفرق بين زمن إكمال العملية ووقت وصولها:

$$TAT_i = CT_i - AT_i$$

9.2 زمن الانتظار

في النموذج الذي يحتوي على دفعة CPU واحدة لكل عملية، يحسب زمن الانتظار بطرح زمن التنفيذ من زمن الدوران:

$$WT_i = TAT_i - BT_i$$

9.3 متوسط زمن الانتظار

$$\text{Average WT} = (\sum WT_i) / n$$

9.4 متوسط زمن الدوران

$$\text{Average TAT} = (\sum TAT_i) / n$$

9.5 استخدام وحدة المعالجة المركزية

$$\text{CPU Utilization} = (\text{CPU Busy Time} / \text{Total Simulation Time}) \times 100$$

حيث يمثل CPU Busy Time مجموع مدد تنفيذ العمليات، ويمثل Total Simulation Time المدة من بداية المحاكاة حتى اكتمال آخر عملية، بما فيها فترات الخمول المسجلة.

9.6 مثال حسابي موجز

إذا وصلت عملية عند الزمن 1، واکتملت عند الزمن 9، وكان زمن تنفيذها 3 وحدات، فإن:

$$TAT = 9 - 1 = 8$$

$$WT = 8 - 3 = 5$$

10. افتراضات المحاكاة وقواعد كسر التعادل

تعتمد المحاكاة الافتراضات الآتية:

1. توجد وحدة معالجة مركزية واحدة.
2. كل عملية تحتوي على دفعة CPU واحدة ولا تحتوي على فترات I/O داخلية.
3. قيم Arrival Time غير سالبة، وقيم Burst Time موجبة.
4. الرقم الأصغر في حقل Priority يمثل أولوية أعلى.

5. FCFS و SJF و Priority منفذة بصيغة غير مستبقة.
6. Round Robin منفذة بصيغة مستبقة وبشريحة زمنية مقدارها وحدتان.
7. لا تُحتسب كلفة التبديل السياقي.
8. يسجل المعالج Idle عندما لا توجد عملية جاهزة.
9. يُكسر التعادل أولاً بوقت الوصول، ثم بترتيب تعريف العملية داخل العينة.
10. تستخدم الخوارزميات مجموعة العمليات نفسها لضمان عدالة المقارنة.

11. التعقيد الزمني والمساحي

نُشئ الدالة `buildArrivalOrder` لترتيب الوصول باستخدام Selection Sort ، ولذلك تستغرق مرحلة الترتيب السابقة للجدولة $O(n^2)$ وبعد اكتمال ترتيب الوصول تكون تعقيدات التنفيذ كما يأتي:

الخوارزمية	تعقيد مرحلة الجدولة بعد الترتيب	الملاحظة
FCFS	$O(n)$	تدخل كل عملية إلى الطابور وتخرج منه مرة واحدة
SJF	$O(n \log n)$	إدخال واستخراج العمليات من الكومة الثنائية
Priority	$O(n \log n)$	إدخال واستخراج العمليات من طابور الأولوية
Round Robin	$O(d)$	حيث d عدد الشرائح المنفذة، وقد يتجاوز عدد العمليات

وعند احتساب مرحلة ترتيب الوصول ضمن كل تشغيل يضاف حد $O(n^2)$ إلى التعقيد السابق. وفي حجم العينة الحالي يظل هذا الأثر محدودًا، بينما يوضح التنفيذ بصورة مباشرة خوارزمية Selection Sort. تحتاج النتائج ومؤشرات العمليات إلى مساحة $O(n)$ ، بينما قد يحتاج سجل جانت إلى مساحة تتناسب مع عدد مقاطع التنفيذ، خصوصًا في Round Robin.

12. نتائج التنفيذ

12.1 نتائج FCFS

البيان	القيمة
ترتيب التنفيذ	$P1 \rightarrow P2 \rightarrow P3 \rightarrow P4 \rightarrow P5$
مخطط جانت	[0-2: IDLE] [2-7: P1] [7-9: P2] [9-17: P3] [17-20: P4] [20-24: P5]
زمن انشغال المعالج	22 وحدة زمنية
زمن حمول المعالج	وحدتان زمنيّتان

العملية	AT	BT	PR	CT	WT	TAT
P1	2	5	3	7	0	5
P2	3	2	1	9	4	6
P3	4	8	4	17	5	13
P4	5	3	2	20	12	15
P5	7	4	1	24	13	17

متوسط زمن الانتظار	متوسط زمن الدوران	استخدام وحدة المعالجة
6.80	11.20	91.67%

12.2 نتائج SJF

البيان	القيمة
ترتيب التنفيذ	P1 → P2 → P4 → P5 → P3
مخطط جانت	[0-2: IDLE] [2-7: P1] [7-9: P2] [9-12: P4] [12-16: P5] [16-24: P3]
زمن انشغال المعالج	22 وحدة زمنية
زمن حمول المعالج	وحدتان زمنيتان

العملية	AT	BT	PR	CT	WT	TAT
P1	2	5	3	7	0	5
P2	3	2	1	9	4	6
P3	4	8	4	24	12	20
P4	5	3	2	12	4	7
P5	7	4	1	16	5	9

متوسط زمن الانتظار	متوسط زمن الدوران	استخدام وحدة المعالجة
5.00	9.40	91.67%

12.3 نتائج Priority Scheduling

البيان	القيمة
ترتيب التنفيذ	P1 → P2 → P5 → P4 → P3
مخطط جانت	[0-2: IDLE] [2-7: P1] [7-9: P2] [9-13: P5] [13-16: P4] [16-24: P3]
زمن انشغال المعالج	22 وحدة زمنية
زمن حمول المعالج	وحدتان زمنيتان

العملية	AT	BT	PR	CT	WT	TAT
P1	2	5	3	7	0	5
P2	3	2	1	9	4	6
P3	4	8	4	24	12	20
P4	5	3	2	16	8	11
P5	7	4	1	13	2	6

متوسط زمن الانتظار	متوسط زمن الدوران	استخدام وحدة المعالجة
5.20	9.60	91.67%

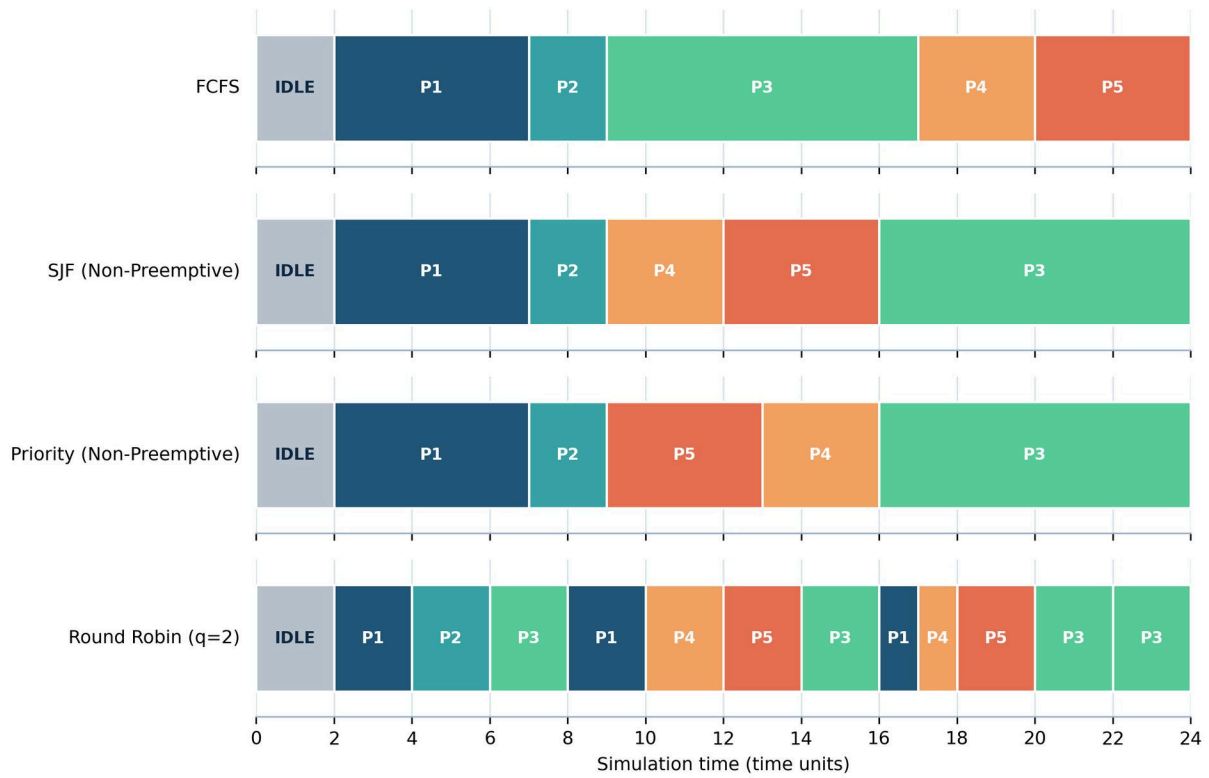
12.4 نتائج Round Robin

البيان	القيمة
قيمة Time Quantum	2
ترتيب التنفيذ	P1 → P2 → P3 → P1 → P4 → P5 → P3 → P1 → P4 → P5 → P3 → P3
مخطط جانت	[0-2: IDLE] [2-4: P1] [4-6: P2] [6-8: P3] [8-10: P1] [10-12: P4] [12-14: P5] [14-16: P3] [16-17: P1] [17-18: P4] [18-20: P5] [20-22: P3] [22-24: P3]
زمن انشغال المعالج	22 وحدة زمنية
زمن خمول المعالج	وحدتان زمنيتان

العملية	AT	BT	PR	CT	WT	TAT
P1	2	5	3	17	10	15
P2	3	2	1	6	1	3
P3	4	8	4	24	12	20
P4	5	3	2	18	10	13
P5	7	4	1	20	9	13

متوسط زمن الانتظار	متوسط زمن الدوران	استخدام وحدة المعالجة
8.40	12.80	91.67%

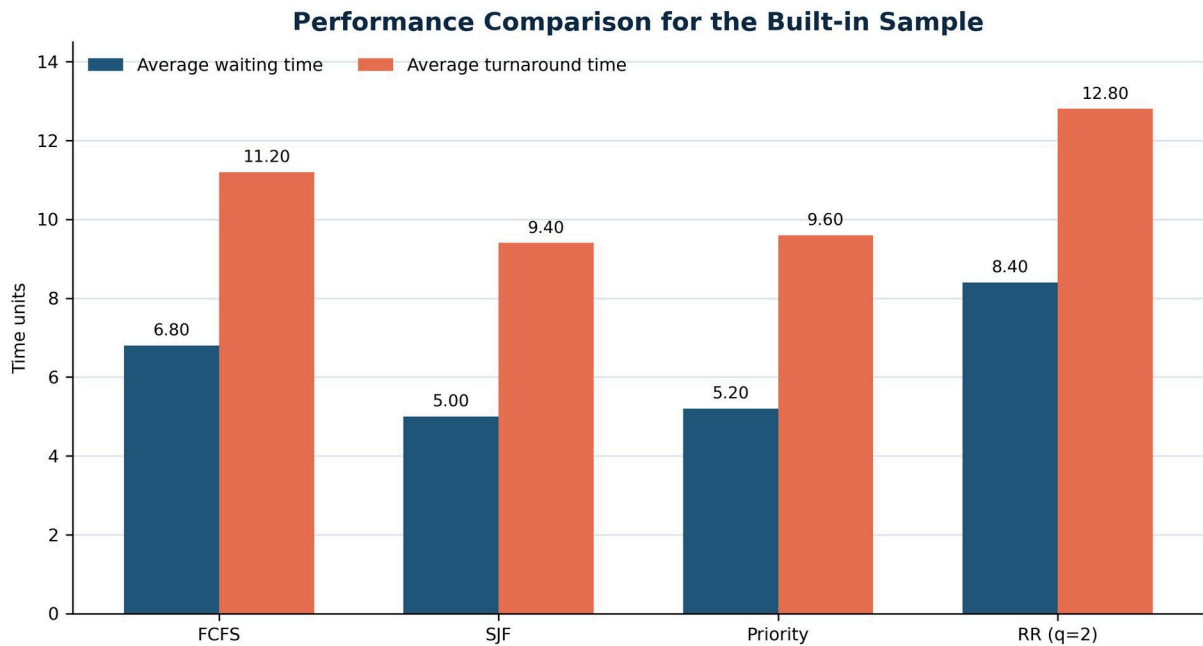
Gantt Charts for the Built-in Sample



شكل (1): مخططات Gantt الناتجة من التشغيل الفعلي للبيئة المدمجة

13. مقارنة الأداء

الخوارزمية	متوسط WT	متوسط TAT	CPU Utilization	طبيعة الجدولة	الهيكل الرئيس
FCFS	6.80	11.20	91.67%	غير مستبقة	Queue
SJF	5.00	9.40	91.67%	غير مستبقة	Priority Queue
Priority	5.20	9.60	91.67%	غير مستبقة	Priority Queue
Round Robin	8.40	12.80	91.67%	مستبقة	Queue



شكل (2): مقارنة متوسط زمن الانتظار ومتوسط زمن الدوران

توضح النتائج أن اختلاف ترتيب التنفيذ ينعكس مباشرة على WT و TAT. وقد تتساوى بعض الخوارزميات في النتائج عندما تقود بيانات الوصول والأزمنة إلى ترتيب تنفيذ واحد. كما قد تتساوى نسبة CPU Utilization بين الخوارزميات الأربع؛ لأن جميعها تعمل على حمل العمل نفسه، ولا تُحتسب كلفة التبديل السياقي، وكل خوارزمية تبقى المعالج مشغولاً متى وُجدت عملية جاهزة. لذلك لا يدل تساوي الاستخدام على تطابق الخوارزميات، بل تُستكمل المقارنة بمتوسطتي الانتظار والدوران وعدالة الاستجابة.

مقياس المقارنة	النتيجة
أقل متوسط زمن انتظار	SJF بقيمة 5.00
أقل متوسط زمن دوران	SJF بقيمة 9.40
أعلى استخدام للمعالج	تعاذلت الخوارزميات الأربع بنسبة 91.67%

14. الاختبار والتحقق

14.1 اختبارات صحة الحساب

أجري التحقق من كل نتيجة وفق العلاقات الآتية:

- CT يساوي نهاية آخر مقطع تنفيذ للعملية.
- TAT يساوي CT ناقص AT.
- WT يساوي TAT ناقص BT.
- مجموع أزمنة التنفيذ الفعلية يساوي مجموع BT لجميع العمليات.
- لا تبدأ أي عملية قبل وقت وصولها.
- لا تنفذ عملية مكتملة مرة أخرى.
- لا يتجاوز مقطع Round Robin قيمة Time Quantum إلا إذا كان المقطع الأخير أقصر منها.

14.2 اختبارات هياكل البيانات

الاختبار	النتيجة المتوقعة	حالة الاختبار
إدخال عناصر إلى Queue وإخراجها	المحافظة على FIFO	ناجح
إدخال عناصر إلى Priority Queue	استخراج العنصر الأعلى حسب قاعدة الخوارزمية	ناجح
إضافة العمليات إلى Linked List	حفظ جميع العمليات بالترتيب	ناجح
دفع مقاطع التنفيذ إلى Stack	إظهار المقاطع بترتيب عكسي	ناجح
الوصول إلى بيانات Array	تطابق القيم مع العينة المدمجة	ناجح

14.3 اختبارات حالات الجدولة

الحالة	غرض الاختبار	حالة الاختبار
بدء العينة قبل وصول أول عملية	التحقق من تسجيل IDLE من 0 إلى 2	ناجح
وجود فجوة بلا عمليات جاهزة	التحقق من تسجيل Idle	ناجح
تبادل أزمنة التنفيذ	التحقق من كسر التعادل بوقت الوصول ثم ترتيب المصفوفة	ناجح
تبادل الأولوية	التحقق من قاعدة التعادل	ناجح
Burst Time أكبر من Quantum	التحقق من إعادة العملية إلى Queue	ناجح
انتهاء العملية قبل Quantum	التحقق من عدم إعادة العملية المكتملة	ناجح

14.4 بيئة البناء

البيان	القيمة
لغة البرمجة	C++
المعيار	C++17
نوع التطبيق	Console Application
أنظمة التشغيل المستهدفة	Linux و Windows
المتجمات المدعومة	MinGW g++، GCC، والمتراجم المتوافق مع C++17 في بيئات التطوير الشائعة

15. تنظيم المشروع على GitHub

يُنظم المستودع وفق البنية الآتية:

```
cpu-scheduling-simulation-ds/
├── src/
│   └── CPU_Scheduling_Simulator.cpp
├── docs/
│   ├── Project_Report_AR.{docx,pdf}
│   ├── Run_Guide_AR.{docx,pdf}
│   └── Code_Explanation_AR.{docx,pdf}
├── output/
│   └── Sample_Output.txt
├── CMakeLists.txt
├── run_linux.sh
├── run_windows.bat
└── README.md, LICENSE, .gitignore
```

يحتوي ملف README.md على وصف مختصر للمشروع، والخوارزميات وهياكل البيانات المنفذة، ومتطلبات البناء، وأوامر التشغيل. ويوفر المستودع سجلًا واضحًا للتعديلات، ويسهل الوصول إلى الكود والتقارير ونتائج التشغيل من رابط واحد.

رابط المستودع المعتمد: <https://github.com/alimohammedabotaleb-hub/cpu-scheduling-simulation-ds>

16. حدود المشروع والتطوير المستقبلي

يمكن توسيع المشروع مستقبلاً ليشمل:

- SRTF بوصفها النسخة المستتقة من SJF.
- النسخة المستتقة من Priority Scheduling.
- احتساب كلفة Context Switch.
- حساب Response Time و Throughput.
- دعم أكثر من مجموعة اختبار مدمجة.
- تصدير النتائج تلقائيًا إلى CSV.
- إضافة واجهة رسومية لعرض مخطط جانت.
- دعم جدولة متعددة الأنوية.

17. الخاتمة

حقق المشروع محاكاة موحدة لأربع خوارزميات أساسية في جدولة وحدة المعالجة المركزية، وربط كل خوارزمية بهيكل البيانات الملائم لطريقة عملها. أظهر التنفيذ أن Queue تمثل الاختيار الطبيعي لسياسات الترتيب والدوران، وأن Priority Queue توفر اختيارًا فعالاً للعملية الأقصر أو الأعلى أولوية، بينما دعمت Array تنظيم البيانات والنتائج، وحفظت Linked List العمليات، ووثق Stack سجل التنفيذ العكسي.

أتاحت المقاييس المحسوبة تقييم النتائج بصورة كمية بدل الاكتفاء بترتيب التنفيذ. وقد بينت المقارنة أن اختيار الخوارزمية يتوقف على الهدف المطلوب؛ فقد يكون تقليل متوسط الانتظار هو الأولوية في سياق، بينما تكون عدالة توزيع وقت المعالج والاستجابة الدورية أهم في سياق آخر. ويقدم التصميم الناتج أساسًا منظمًا وقابلًا للتطوير لدراسة خوارزميات إضافية وقياسات أوسع.

18. المراجع

المرجع	الرقم
Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, 9th Edition, Wiley, 2012, Chapter 6: CPU Scheduling.	1
B. Stroustrup, Programming: Principles and Practice Using C++, 2nd Edition, Addison-Wesley, 2014.	2
ISO/IEC 14882, Programming Languages - C++, C++17 Standard.	3

الملحق (أ): اختصارات النتائج

الاختصار	المصطلح الإنجليزي	المقابل العربي
PID	Process ID	معرف العملية
AT	Arrival Time	وقت الوصول
BT	Burst Time	زمن التنفيذ
PR	Priority	الأولوية
CT	Completion Time	زمن الإكمال
WT	Waiting Time	زمن الانتظار
TAT	Turnaround Time	زمن الدوران
CPU	Central Processing Unit	وحدة المعالجة المركزية
RR	Round Robin	التناوب الدائري

الملحق (ب): بيانات التسليم

البيان	القيمة
اسم المشروع	CPU Scheduling Simulator Using Appropriate Data Structures
لغة التنفيذ	C++17
الخوارزميات	FCFS , SJF , Priority Scheduling , Round Robin
هياكل البيانات	Array , Queue , Priority Queue , Linked List , Stack
المقاييس	WT , TAT , Average WT , Average TAT , CPU Utilization
طريقة البيانات	عينة مدمجة وتشغيل تلقائي دون إدخال يدوي
مستودع GitHub	https://github.com/alimohammedabotaleb-hub/cpu-scheduling-simulation-ds
إعداد	مؤمن عبدالله الشامي
إشراف	د. بلال مرشد